Optimizing SQL (and DataFrames) in DataFusion, Part 2: Optimizers in Apache DataFusion

Posted on: Sun 15 June 2025 by alamb, akurmustafa

Contents

Note, this blog was originally published on the InfluxData blog.

In the first part of this post, we discussed what a Query Optimizer is, what role it plays, and described how industrial optimizers are organized. In this second post, we describe various optimizations that are found in Apache DataFusion and other industrial systems in more

detail.

DataFusion contains high quality, full-featured implementations for *Always Optimizations* and *Engine Specific Optimizations* (defined in Part 1). Optimizers are implemented as rewrites of `LogicalPlan` in the logical optimizer or rewrites of `ExecutionPlan` in the physical optimizer. This design means the same optimizer passes are applied for SQL queries, DataFrame queries, as well as plans for other query language frontends such as InfluxQL in InfluxDB 3.0, PromQL in Greptime, and vega in VegaFusion.

Always Optimizations

Some optimizations are so important they are found in almost all query engines and are typically the first implemented as they provide the largest cost / benefit ratio (and performance is terrible without them).

Predicate/Filter Pushdown

Why: Avoid carrying unneeded *rows* as soon as possible

What: Moves filters "down" in the plan so they run earlier during execution, as shown in Figure 1.

Example Implementations: DataFusion, DuckDB, ClickHouse

The earlier data is filtered out in the plan, the less work the rest of the plan has to do. Most mature databases aggressively use filter pushdown / early filtering combined with techniques such as partition and storage pruning (e.g. Parquet Row Group pruning) for performance.

An extreme, and somewhat contrived, is the query

```
SELECT city, COUNT(*) FROM population GROUP BY city HAVING city = 'BOSTON';
```

Semantically, `HAVING` is evaluated after `GROUP BY` in SQL. However, computing the population of all cities and discarding everything except Boston is much slower than only computing the population for Boston and so most Query Optimizers will evaluate the filter before the aggregation.

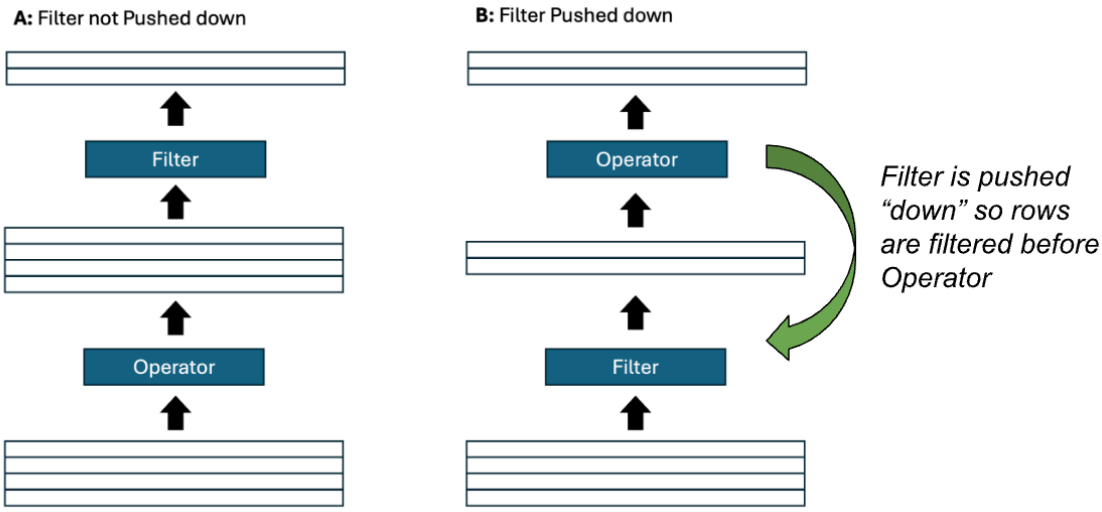

Figure 1: Filter Pushdown. In (**A**) without filter pushdown, the operator processes more rows, reducing efficiency. In (**B**) with filter pushdown, the operator receives fewer rows, resulting in less overall work and leading to a faster and more efficient query.

Projection Pushdown

Why: Avoid carrying unneeded *columns* as soon as possible

What: Pushes "projection" (keeping only certain columns) earlier in the plan, as shown in Figure 2.

Example Implementations: Implementations: DataFusion, DuckDB, ClickHouse

Similarly to the motivation for *Filter Pushdown*, the earlier the plan stops doing something, the less work it does overall and thus the faster it runs. For Projection Pushdown, if columns are not needed later in a plan, copying the data to the output of other operators is unnecessary and the costs of copying can add up. For example, in Figure 3 of Part 1, the `species` column is only needed to evaluate the Filter within the scan and `notes` are never used, so it is unnecessary to copy them through the rest of the plan.

Projection Pushdown is especially effective and important for column store databases, where the storage format itself (such as Apache Parquet) supports efficiently reading only a subset of required columns, and is especially powerful in combination with filter pushdown. Projection Pushdown is still important, but less effective for row oriented formats such as JSON or CSV where each column in each row must be parsed even if it is not used in the plan.

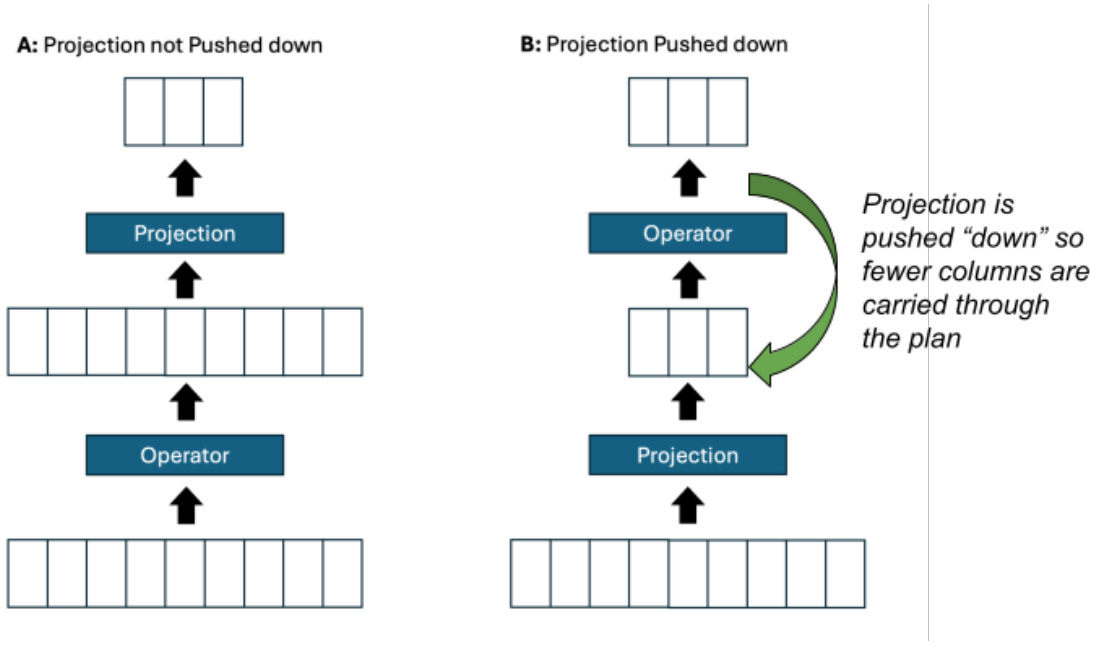

Figure 2: In (**A**) without projection pushdown, the operator receives more columns, reducing efficiency. In (**B**) with projection pushdown, the operator receives fewer columns, leading to optimized execution.

Limit Pushdown

Why: The earlier the plan stops generating data, the less overall work it does, and some operators have more efficient limited implementations.

What: Pushes limits (maximum row counts) down in a plan as early as possible.

Example Implementations: DataFusion, DuckDB, ClickHouse, Spark (Window and Projection)

Often queries have a `LIMIT` or other clause that allows them to stop generating results early so the sooner they can stop execution, the more efficiently they will execute.

In addition, DataFusion and other systems have more efficient implementations of some operators that can be used if there is a limit. The classic example is replacing a full sort + limit with a TopK operator that only tracks the top values using a heap. Similarly, DataFusion's Parquet reader stops fetching and opening additional files once the limit has been hit.

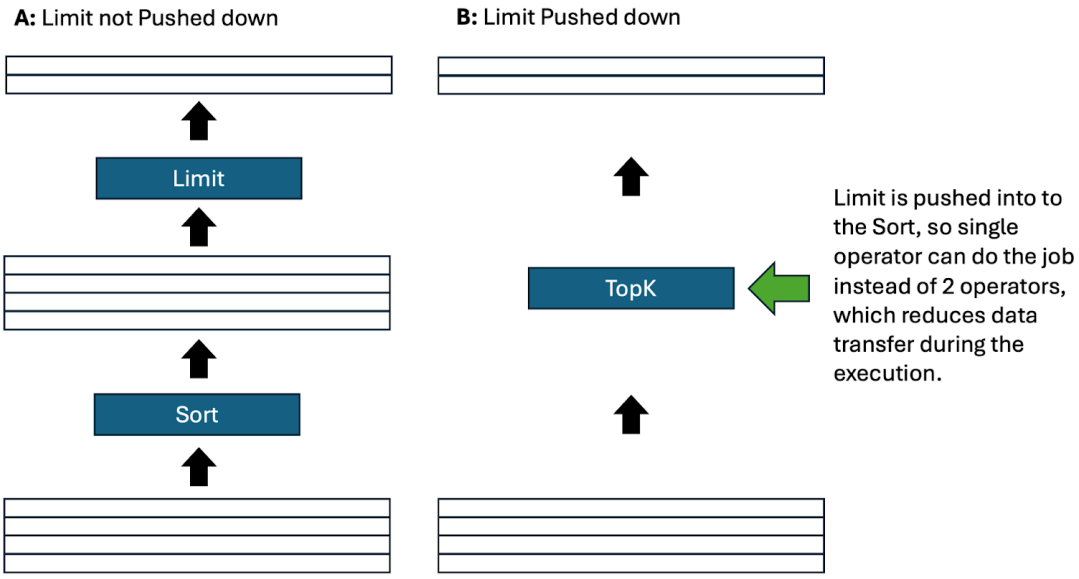

Figure 3: In (**A**), without limit pushdown all data is sorted and everything except the first few rows are discarded. In (**B**), with limit pushdown, Sort is replaced with TopK operator which does much less work.

Expression Simplification / Constant Folding

Why: Evaluating the same expression for each row when the value doesn't change is wasteful.

What: Partially evaluates and/or algebraically simplify expressions.

Example Implementations: DataFusion, DuckDB (has several rules such as constant folding, and comparison simplification), Spark

If an expression doesn't change from row to row, it is better to evaluate the expression **once** during planning. This is a classic compiler technique and is also used in database systems

For example, given a query that finds all values from the current year

```
SELECT … WHERE extract(year from time_column) = extract(year from now())
```

Evaluating `extract(year from now())` on every row is much more expensive than evaluating it once during planning time so that the query becomes comparison to a constant

```
SELECT … WHERE extract(year from time_column) = 2025
```

Furthermore, it is often possible to push such predicates **into** scans.

Rewriting `OUTER JOIN` → `INNER JOIN`

Why: `INNER JOIN` implementations are almost always faster (as they are simpler) than `OUTER JOIN` implementations, and `INNER JOIN` s impose fewer restrictions on other optimizer passes (such as join reordering and additional filter pushdown).

What: In cases where it is known that NULL rows introduced by an `OUTER JOIN` will not appear in the results, it can be rewritten to an `INNER JOIN`.

Example Implementations: DataFusion, Spark, ClickHouse.

For example, given a query such as the following

```
SELECT …
FROM orders LEFT OUTER JOIN customer ON (orders.cid = customer.id)
WHERE customer.last_name = 'Lamb'
```

The `LEFT OUTER JOIN` keeps all rows in `orders` that don't have a matching customer, but fills in the fields with `null`. All such rows will be filtered out by `customer.last_name = 'Lamb'`, and thus an INNER JOIN produces the same answer. This is illustrated in Figure 4.

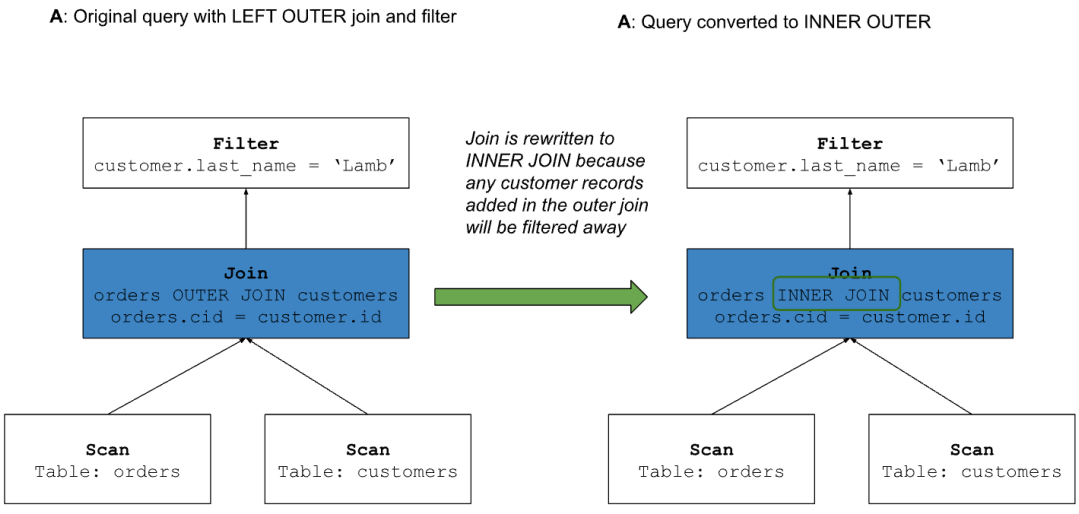

Figure 4: Rewriting `OUTER JOIN` to `INNER JOIN`. In (A) the original query contains an `OUTER JOIN` but also a filter on `customer.last_name`, which filters out all rows that might be introduced by the `OUTER JOIN`. In (B) the `OUTER JOIN` is converted to inner join, a more

efficient implementation can be used.

Engine Specific Optimizations

As discussed in Part 1 of this blog, optimizers also contain a set of passes that are still always good to do, but are closely tied to the specifics of the query engine. This section describes some common types

Subquery Rewrites

Why: Actually implementing subqueries by running a query for each row of the outer query is very expensive.

What: It is possible to rewrite subqueries as joins which often perform much better.

Example Implementations: DataFusion (one, two, three), Spark

Evaluating subqueries a row at a time is so expensive that execution engines in high performance analytic systems such as DataFusion and Vertica may not even support row-at-a-time evaluation given how terrible the performance would be. Instead, analytic systems rewrite such queries into joins which can perform 100s or 1000s of times faster for large datasets. However, transforming subqueries to joins requires "exotic" join semantics such as `SEMI JOIN`, `ANTI JOIN` and variations on how to treat equality with null[7].

For a simple example, consider that a query like this:

```
SELECT customer.name
FROM customer
WHERE (SELECT sum(value)
       FROM orders WHERE
       orders.cid = customer.id) > 10;
```

Can be rewritten like this:

```
SELECT customer.name
FROM customer
JOIN (
  SELECT customer.id as cid_inner, sum(value) s
  FROM orders
  GROUP BY customer.id
```

```
  ) ON (customer.id = cid_inner AND s > 10);
```

We don't have space to detail this transformation or why it is so much faster to run, but using this and many other transformations allow efficient subquery evaluation.

Optimized Expression Evaluation

Why: The capabilities of expression evaluation vary from system to system.

What: Optimize expression evaluation for the particular execution environment.

Example Implementations: There are many examples of this type of optimization, including DataFusion's Common Subexpression Elimination, unwrap_cast, and identifying equality join predicates. DuckDB rewrites IN clauses, and SUM expressions. Spark also unwraps casts in binary comparisons, and adds special runtime filters.

To give a specific example of what DataFusion's common subexpression elimination does, consider this query that refers to a complex expression multiple times:

```
SELECT date_bin('1 hour', time, '1970-01-01')
FROM table
WHERE date_bin('1 hour', time, '1970-01-01') >= '2025-01-01 00:00:00'
ORDER BY date_bin('1 hour', time, '1970-01-01')
```

Evaluating `date_bin('1 hour', time, '1970-01-01')` each time it is encountered is inefficient compared to calculating its result once, and reusing that result in when it is encountered again (similar to caching). This reuse is called *Common Subexpression Elimination*.

Some execution engines implement this optimization internally to their expression evaluation engine, but DataFusion represents it explicitly using a separate Projection plan node, as illustrated in Figure 5. Effectively, the query above is rewritten to the following

```
SELECT time_chunk
FROM(SELECT date_bin('1 hour', time, '1970-01-01') as time_chunk
     FROM table)
WHERE time_chunk >= '2025-01-01 00:00:00'
ORDER BY time_chunk
```

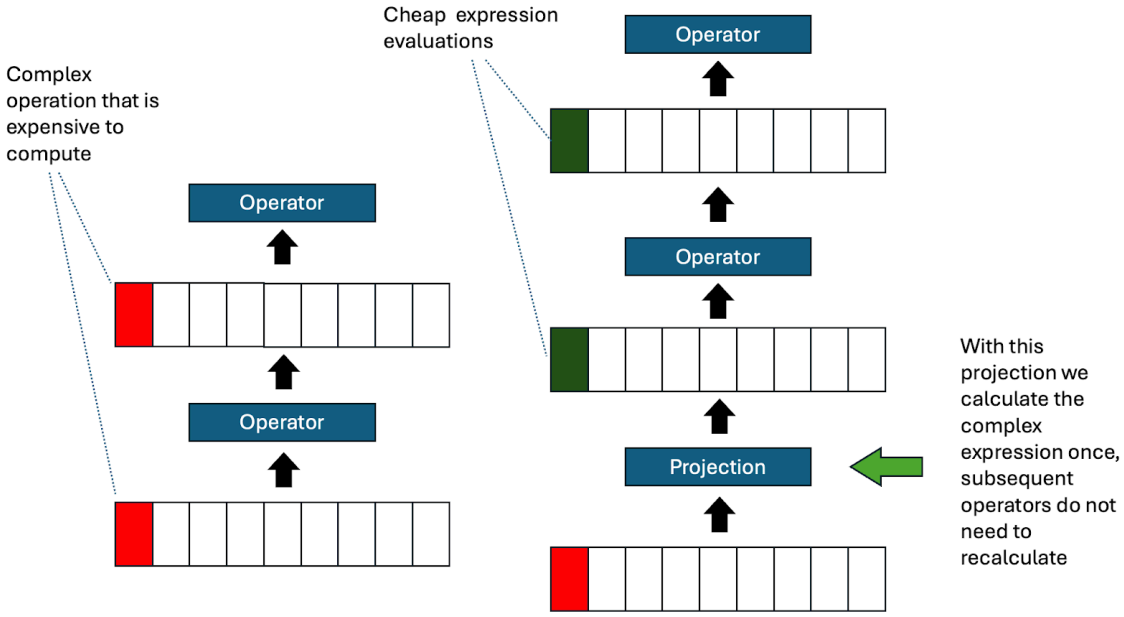

Figure 5: Adding a Projection to evaluate common complex sub expression decreases complexity for later stages.

Algorithm Selection

Why: Different engines have different specialized operators for certain operations.

What: Selects specific implementations from the available operators, based on properties of the query.

Example Implementations: DataFusion's EnforceSorting pass uses sort optimized implementations, Spark's rewrite to use a special operator for ASOF joins, and ClickHouse's join algorithm selection such as when to use MergeJoin

For example, DataFusion uses a `TopK` (source) operator rather than a full `Sort` if there is also a limit on the query. Similarly, it may choose to use the more efficient `PartialOrdered` grouping operation when the data is sorted on group keys or a `MergeJoin`

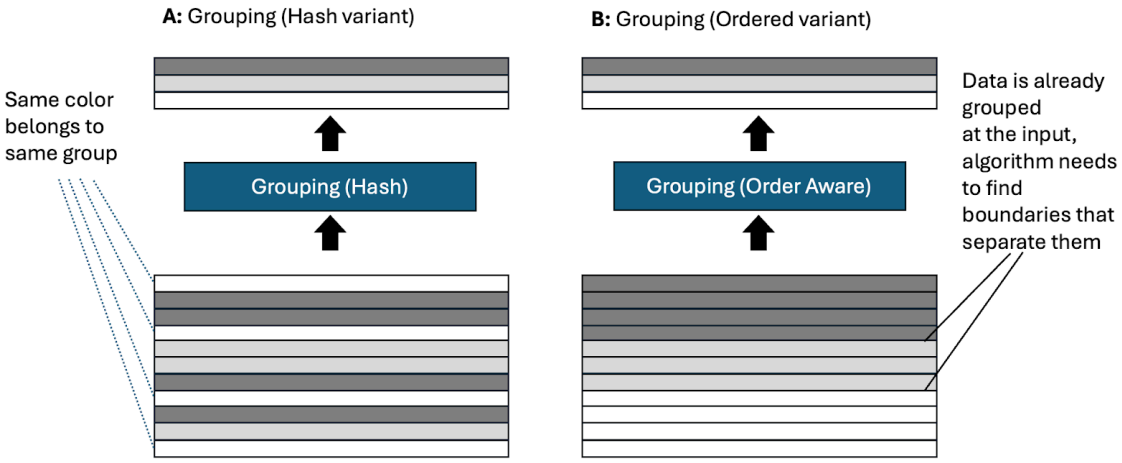

Figure 6: An example of specialized operation for grouping. In (**A**), input data has no specified ordering and DataFusion uses a hashing-based grouping operator (source) to determine distinct groups. In (**B**), when the input data is ordered by the group keys, DataFusion uses a specialized grouping operator (source) to find boundaries that separate groups.

Using Statistics Directly

Why: Using pre-computed statistics from a table, without actually reading or opening files, is much faster than processing data.

What: Replace calculations on data with the value from statistics.

Example Implementations: DataFusion, DuckDB,

Some queries, such as the classic `COUNT(*) from my_table` used for data exploration can be answered using only statistics. Optimizers often have access to statistics for other reasons (such as Access Path and Join Order Selection) and statistics are commonly stored in analytic file formats. For example, the Metadata of Apache Parquet files stores `MIN`, `MAX`, and `COUNT` information.

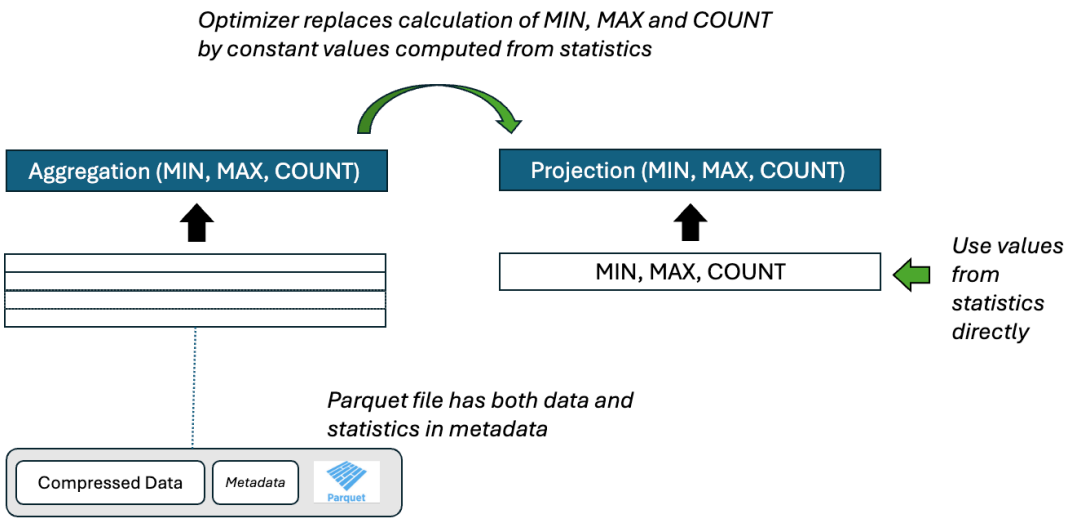

Figure 7: When the aggregation result is already stored in the statistics, the query can be evaluated using the values from statistics without looking at any compressed data. The optimizer replaces the Aggregation operation with values from statistics.

Access Path and Join Order Selection

Overview

Last, but certainly not least, are optimizations that choose between plans with potentially (very) different performance. The major options in this category are

1. **Join Order:** In what order to combine tables using JOINs?
2. **Access Paths:** Which copy of the data or index should be read to find matching tuples?
3. [Materialized View](): Can the query can be rewritten to use a materialized view (partially computed query results)? This topic deserves its own blog (or book) and we don't discuss further here.

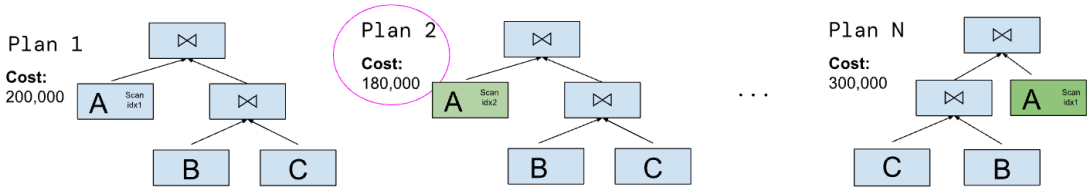

Figure 8: Access Path and Join Order Selection in Query Optimizers. Optimizers use heuristics to enumerate some subset of potential join orders (shape) and access paths (color). The plan with the smallest estimated cost according to some cost model is chosen.

In this case, Plan 2 with a cost of 180,000 is chosen for execution as it has the lowest estimated cost.

This class of optimizations is a hard problem for at least the following reasons:

1. **Exponential Search Space**: the number of potential plans increases exponentially as the number of joins and indexes increases.
2. **Performance Sensitivity**: Often different plans that are very similar in structure perform very differently. For example, swapping the input order to a hash join can result in 1000x or more (yes, a thousand-fold!) run time differences.
3. **Cardinality Estimation Errors**: Determining the optimal plan relies on cardinality estimates (e.g., how many rows will come out of each join). It is a known hard problem to estimate this cardinality, and in practice queries with as few as 3 joins often have large cardinality estimation errors.

Heuristics and Cost-Based Optimization

Industrial optimizers handle these problems using a combination of

1. **Heuristics:** to prune the search space and avoid considering plans that are (almost) never good. Examples include considering left-deep trees, or using `Foreign Key` / `Primary Key` relationships to pick the build size of a hash join.
2. **Cost Model**: Given the smaller set of candidate plans, the Optimizer then estimates their cost and picks the one using the lowest cost.

For some examples, you can read about Spark's cost-based optimizer or look at the code for DataFusion's join selection and DuckDB's cost model and join order enumeration.

However, the use of heuristics and (imprecise) cost models means optimizers must

1. **Make deep assumptions about the execution environment:** For example the heuristics often include assumptions that joins implement sideways information passing (RuntimeFilters), or that Join operators always preserve a particular input's order.
2. **Use one particular objective function:** There are almost always trade-offs between desirable plan properties, such as execution speed, memory use, and robustness in the face of cardinality estimation. Industrial optimizers typically have one cost function which attempts to balance between the properties or a series of hard to use indirect

tuning knobs to control the behavior.

3. **Require statistics**: Typically cost models require up-to-date statistics, which can be expensive to compute, must be kept up to date as new data arrives, and often have trouble capturing the non-uniformity of real world datasets

Join Ordering in DataFusion

DataFusion purposely does not include a sophisticated cost based optimizer. Instead, keeping with its design goals it provides a reasonable default implementation along with extension points to customize behavior.

Specifically, DataFusion includes

1. "Syntactic Optimizer" (joins in the order they are listed in the query[8]) with basic join re-ordering (source) to prevent join disasters.
2. Support for ColumnStatistics and Table Statistics
3. The framework for filter selectivity + join cardinality estimation.
4. APIs for easily rewriting plans, such as the TreeNode API and reordering joins

This combination of features along with custom optimizer passes lets users customize the behavior to their use case, such as custom indexes like uWheel and materialized views.

The rationale for including only a basic optimizer is that any one particular set of heuristics and cost model is unlikely to work well for the wide variety of DataFusion users because of the tradeoffs involved.

For example, some users may always have access to adequate resources, and want the fastest query execution, and are willing to tolerate runtime errors or a performance cliff when there is insufficient memory. Other users, however, may be willing to accept a slower maximum performance in return for more predictable performance when running in a resource constrained environment. This approach is not universally agreed. One of us has previously argued the case for specialized optimizers in a more academic paper, and the topic comes up regularly in the DataFusion community, (e.g. this recent comment).

Note: We are actively improving this part of the code to help people write their own optimizers (🎣 come help us define and implement it!)

Conclusion

Optimizers are awesome, and we hope these two posts have demystified what they are and how they are implemented in industrial systems. Like many modern query engine designs, the common techniques are well known, though require substantial effort to get right. DataFusion's industrial strength optimizers can and do serve many real world systems well and we expect that number to grow over time.

We also think DataFusion provides interesting opportunities for optimizer research. As we discussed, there are still unsolved problems such as optimal join ordering. Experiments in papers often use academic systems or modify optimizers in tightly integrated open source systems (for example, the recent POLARs paper uses DuckDB). However, using a tightly integrated system constrains the research to the set of heuristics and structure provided by that system. Hopefully DataFusion's documentation, newly citeable SIGMOD paper, and modular design will encourage more broadly applicable research in this area.

And finally, as always, if you are interested in working on query engines and learning more about how they are designed and implemented, please join our community. We welcome first time contributors as well as long time participants to the fun of building a database together.

Notes

[7] See Unnesting Arbitrary Queries from Neumann and Kemper for a more academic treatment.

[8] One of my favorite terms I learned from Andy Pavlo's CMU online lectures

---

Comments

We use Giscus for comments, powered by GitHub Discussions. To respect your privacy, Giscus and comments will load only if you click "Show Comments"

Show Comments

Copyright 2026, The Apache Software Foundation, Licensed under the Apache License, Version 2.0.
Apache® and the Apache feather logo are trademarks of The Apache Software Foundation.